A Comparative Study of Audio Feature Extraction and Machine Learning Methods for Bird Species Classification*

Kryspin Dziarek[1], Szymon Pająk[1] and Marcin Adamski[1]

[1]*Faculty of Applied Mathematics, Silesian University of Technology, Kaszubska 23, 44100 Gliwice, POLAND*

Abstract
wstep - 1 akapit, co zrobiono, jakie wyniki, minimum 180 słów

Keywords
Sound analysis, Birdsong, MFCC (Mel-Frequency Cepstral Coefficients), Random forest, Neural network

1. Introduction

Audio analysis is a process of examining and measuring sound signals to gain information, it has been a subject of research for many decades[1], and now, in the age of artificial intelligence, it has many more applications. Machine learning has become an essential tool in audio signal processing, enabling the development of systems capable of analyzing and classifying sounds. Such systems are widely used in applications including speech recognition[2], music[3] and animal identification[4], often outperforming human perception of acoustic patterns[5].

There are many techniques for feature extraction from sound, one of the most commonly used being Mel-Frequency Cepstral Coefficients (MFCC). This method converts the raw audio signal into a set of coefficients representing the spectral characteristics of sound[6]. Previous studies have established standard configurations for this algorithm, such as the number of Mel filters and cepstral coefficients. However, optimal values depend on specific dataset and application, such as detection of breathing phase[7] or respiratory diseases[8].

In addition to sound recognition, various machine learning algorithms have been applied to audio classification tasks, each providing different levels of performance. Simpler algorithms, such as k-nearest neighbors and random forests, are faster, but often less accurate solutions, while the complex ones, like neural networks, provide higher accuracy at the cost of increased computational complexity.

SSI, project 2025/2026
kd315963@student.polsl.pl (K. Dziarek); sp315960@student.polsl.pl (S. Pająk); ma315929@student.polsl.pl (M. Adamski)
© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).
CEUR Workshop Proceedings (CEUR-WS.org)

This paper presents bird species recognition with sound analysis (using MFCC) and compares different approaches for this system. The sound analysis algorithm consists of many stages, two of them are especially important, because the values they provide depend on chosen data. The number of filters used to describe the frames and number of compressed coefficients strongly affects the performance of the result. Too many features may introduce noise and redundancy, while too few may lead to information loss. The experiments conducted on a common dataset aim to identify the most effective approach for extracting the MFCC features and developing simple classification models with the best performance.

2. Methodology

This section outlines the approach used to compare different methods of audio feature extraction and the development of machine learning models for identifying bird species.

2.1. Audio Feature Extraction

Audio feature extraction enables the transformation of birdsong recordings into numerical representations, which are crucial for training machine learning models. A custom library called *SoundFeatures* was developed to implement the full extraction pipeline. This section describes its structure and role within the experimental setup.

2.1.1. Audio Format Conversion

Audio files were converted from MP3 to FLAC format to provide a consistent and lossless compression of the signal. Although this conversion does not restore the data lost during MP3 compression, it eliminates decoding artifacts and ensures stable input for further signal processing [9].

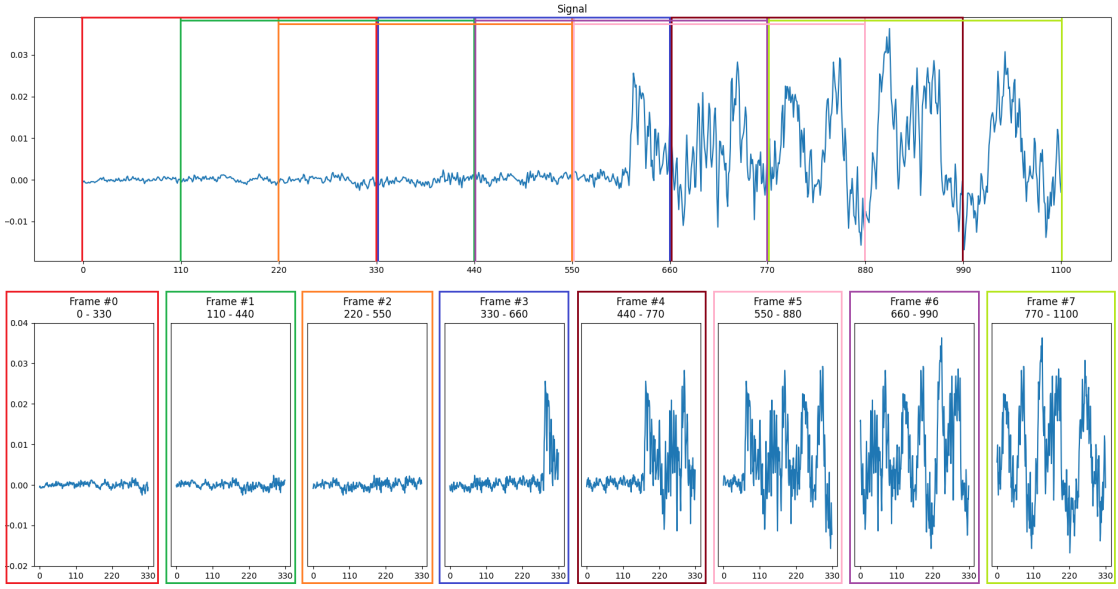
2.1.2. Framing

Framing process consists of dividing the signal into smaller pieces. The resulting segments overlap to reduce the risk of cutting important parts, which could lead to data loss. The framing algorithm used in class had a default frame size set to 25ms and hop size set to 10ms. The algorithm extracts frames using a sliding window of fixed length across the entire signal. Every step has a constant value equal to hop size.

Algorithm 1: Framing method

```
CLASS SoundFeatures
    METHOD frame_signal(signal, frame_size, hop_size)
        frames = []
        for i in range(0, lenghtOf(signal) - frame_size, hop_size) do
            frames.append(signal[i:i + frame_size])
        return frames
    ⋮
```

Figure 1: Visual interpretation of signal framing (hop size set to 110ms and frame size set to 330ms)



2.1.3. Windowing

Windowing involves applying a function to each frame to taper its edges, which minimizes a phenomenon known as spectral leakage. This effect occurs during the Discrete Fourier Transform (DFT), where the frequency components of a frame spread across multiple frequency bins, reducing the accuracy of spectral representation. Various window functions can be used, with the Hamming and Hann windows being among the most common in audio analysis. A rectangular window can also be applied, but it does not modify the signal, resulting in no reduction of spectral leakage[10]. The Hamming and Hann windows differ in their behaviour at the boundaries: the Hann window tapers to zero at both ends, while the Hamming window retains small non-zero values (equal to 0.08 at the boundaries), resulting in less attenuation at the edges. The window functions are defined as follows:

- Hamming window

$$w_{\text{Hamming}}[k] = 0.54 - 0.46 \cos\left(\frac{2\pi k}{N-1}\right)$$

- Hann window

$$w_{\text{Hann}}[k] = 0.5 \cdot \left(1 - \cos\left(\frac{2\pi k}{N-1}\right)\right)$$

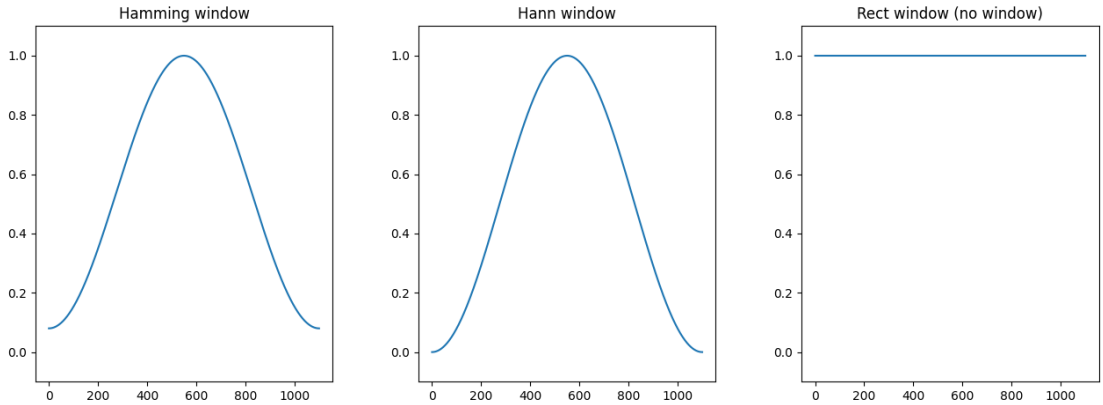
- Rectangular window

$$w_{\text{Rect}}[k] = 1$$

where:

- N - window length
- k - sample index

Figure 2: Window charts



2.1.4. Transformation

Audio analysis uses various mathematical transformations to convert time-domain audio into suitable representations for feature extraction (these representations are also suitable for visualization and manipulation)[11, 12]. Experiments included in this paper focus on frequency domain transformation, specifically on Discrete Fourier Transform (DFT). It maps a signal frame to a set of amplitudes using the following formula:

$$A_k = \sum_{p=0}^{n-1} a_p e^{-\frac{i2\pi kp}{n}}$$

where:

- i - imaginary unit
- k - frequency bin index
- n - sample count in frame a (frame length)
- p - sample index in the time domain
- a_p - sample at index p in frame a

However, the DFT's computational complexity reaches $\mathcal{O}(n^2)$, which results in high computational cost (approximately 10 hours on the examined dataset of 21374 audio files). That is why the implementation of Fast Fourier Transform (FFT) is crucial for analysing bigger datasets. FFT is an algorithm which consists of decomposing the DFT into smaller stages. There are lots of different FFT algorithms[13, 14], the one implemented in this paper is called *Radix-2 Cooley-Tukey*. This method exploits the symmetry of the DFT by separating the signal into components with even and odd indexes, then computing their transforms separately and combining the results using complex exponential coefficients known as twiddle factors[15].

Bit-reversal permutation is used to reorder the input sequence, which enables in-place computing of FFT without additional memory allocation. The algorithm combines pairs of elements at each stage using butterfly operations, which ensures constructing the final frequency-domain representation of the signal with reduced computational complexity ($\mathcal{O}(N \log_2 N)$).

Implementation in SoundFeatures class was divided into fft_frames method, which processes individual frames, and fft method, which computes the transform of each frame.

Algorithm 2: Fast Fourier Transform methods

```
CLASS SoundFeatures
    ⋮
    METHOD fft_frames(frames)
        result = []
        N_samples = lengthOf(frames[0])
        N_fft = 2^⌈log_2 N_samples⌉
        for frame in frames do
            padded_frame = array of N_fft zeros
            padded_frame[0, N_samples] = frame
            result.append(SoundFeatures.fft(padded_frame))
        return result

    METHOD fft(x)
        x = SoundFeatures.bit_reverse(x)
        n = lengthOf(x)
        if n == 1 then
            return x
        else if n % 2 != 0 then
            raise ValueError("The number of samples in frame must be the power of 2")
        s = 2
        for s in range(1, log_2 n + 1) do
            m = 2^s
            half = m integer divided by 2
            w_m = e^(−2i·π/m)
            for k in range(0, n) jumping by m do
                w = 1
                for i in range(0, half) do
                    a = w · x[k + i + half]
                    b = x[k + i]
                    x[k + i] = b + a
                    x[k + i + half] = b − a
                    w = w · w_m
        return x
    ⋮
```

Algorithm 3: Bit-reversal method

```
CLASS SoundFeatures
    ⋮
    METHOD bit_reverse(x)
        n = lengthOf(x)
        k = 0
        for i in range (1, n) do
            bit = n shifted bitwise right by 1
            while k bitwise AND bit != 0 do
                k = k bitwise XOR bit
                bit = bit shifted bitwise right by 1
            k = bitwise OR bit
            if i < k then
                Swap x[i] and x[k]
        return x
    ⋮
```

Even though the FFT algorithm was implemented, experiments included in this paper use numpy, an open-source python library to apply Fast Fourier Transform. The primary reason is improved performance, as numpy is implemented in C, which reduces the overhead associated Python loops, dynamic type checking etc.[16].
In audio feature extraction, FFT is crucial, as it enables efficient conversion of time-domain frames into their frequency-domain representations.

2.1.5. Spectrum

2.1.6. Mel Filterbank

2.1.7. Mel Energies

2.1.8. Mel-Frequency Cepstral Coefficients

2.2. Machine Learning Algorithms

3. Experiments

4. Conclusion

References

[1] J. D. Hollabaugh, Methods for phonemic recognition in speech processing (1963).

[2] M. Zhu, C. Wang, A systematic review of research on ai in language education: Current status and future implications (2025).

[3] S. Serrano, M. A. B. Sahbudin, C. Chaouch, M. Scarpa, A new fingerprint definition for effective song recognition, Pattern Recognition Letters 160 (2022) 135–141.
[4] C. M. Wood, A. Barceinas Cruz, S. Kahl, Pairing a user-friendly machine-learning animal sound detector with passive acoustic surveys for occupancy modeling of an endangered primate, American Journal of Primatology 85 (2023) 269–307.
[5] C. Spille, B. Kollmeier, B. T. Meyer, Comparing human and automatic speech recognition in simple and complex acoustic scenes, Computer Speech & Language 52 (2018) 123–140.
[6] Z. K. Abdul, A. K. Al-Talabani, Mel frequency cepstral coefficient and its applications: A review, Ieee Access 10 (2022) 122136–122158.
[7] A. K. Zhantleuova, Y. K. Makashev, N. T. Duzbayev, Optimizing mfcc parameters for breathing phase detection, Sensors 25 (2025) 5002.
[8] Y. Yan, S. O. Simons, L. van Bemmel, L. G. Reinders, F. M. Franssen, V. Urovi, Optimizing mfcc parameters for the automatic detection of respiratory diseases, Applied Acoustics 228 (2025) 110299.
[9] G. Shi, Data compression for audio-based smart beekeeping (2024) 50–52.
[10] O. K. Hamid, Frame blocking and windowing speech signal, Journal of Information, Communication, and Intelligence Systems (JICIS) 4 (2018) 87–94.
[11] M. Yustisar, P. Sihombing, S. Efendi, Discrete sine transform analysis, discrete cosine transform and discrete fourier transform for introduction to voice register, Int. J. Res. Rev 7 (2020) 155–162.
[12] G. Tzanetakis, G. Essl, P. Cook, Audio analysis using the discrete wavelet transform, in: Proc. conf. in acoustics and music theory applications, volume 66, Citeseer, 2001.
[13] P. Heckbert, Fourier transforms and the fast fourier transform (fft) algorithm, Computer Graphics 2 (1995) 15–463.
[14] M. Soni, P. Kunthe, A general comparison of fft algorithms, Pioneer Journal Of IT & Management (2011).
[15] D. F. Coelho, R. J. Cintra, Discrete fourier transform approximations based on the cooley-tukey radix-2 algorithm, arXiv preprint arXiv:2402.16225 (2024).
[16] M. R. Muqri, E. J. Wilson, J. Shakib, A taste of python–discrete and fast fourier transforms, in: 2015 ASEE Annual Conference & Exposition, 2015, pp. 26–123.